Cat Toy

Elise McDonald
Shealyn Keelen
Rosey Kelly

Date of Report: May 4, 2026

Contents

1 Introduction and Overview

The objective of this project was to design and implement a laser cat toy with randomized laser movements with an Arduino. The system integrates multiple components including two servo motors, a laser module, and IR receiver, and an IR remote.

This project focuses on taking a remote controlled input and combining it with automated servo motor movement to create the laser system. The system must take commands from the IR remote and be able to control the movement speed of the laser. This requires the Arduino to be constantly processing IR signals to dictate the motion of both servo motors.

At a high level, the system operates by receiving commands from the IR remote to determined the speed of the laser system. Different buttons on the remote allow the user to pick slow, medium, or fast movement patterns. When the mode is selected, the Arduino generates random position values with servo angle limits. The system continuously updates the position of the laser at different time intervals depending on which mode was selected.

2 System Overview

The system consists of three main components. IR receiver provides input data from the remote that lets the user select different operating modes. The signal is sent to the Arduino and a random pattern is generated for the laser. The servo motors are the output devices that control the position of the laser. A flowchart of the systems operation is shown in figure 1.

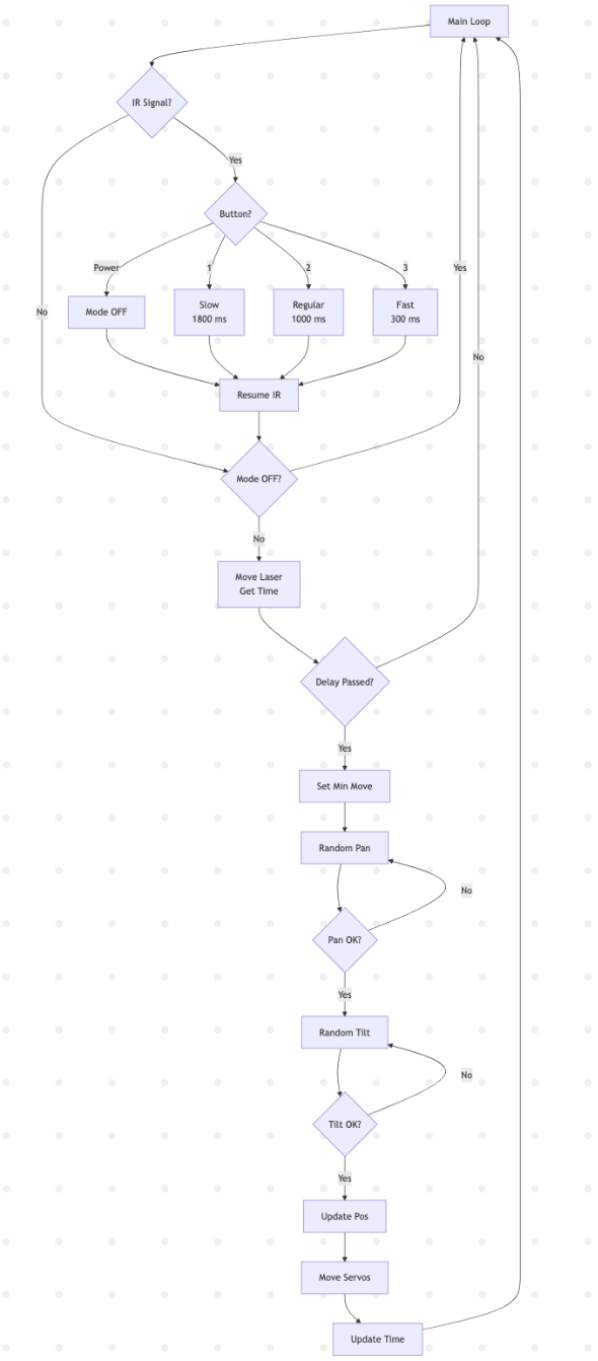

Figure 1: Cay Toy Laser System Flow Chart

The system begins OFF and waits for the signal from the IR remote. When a button on the remote is pressed, the Arduino takes the command and moves based on the user input. The servo motors move the laser to a new position. This process repeats until the user turns the system off on the remote.

Overall, the system follows an input, processing, and output structure. The IR remote acts as the user input, the Arduino controls the logic of the movement commands, and the servo motors control the physical movement.

2.1 Sensing Elements

The sensing element in this project is the IR receiver and remote system. The pinout for the IR receiver is shown in figure 2.

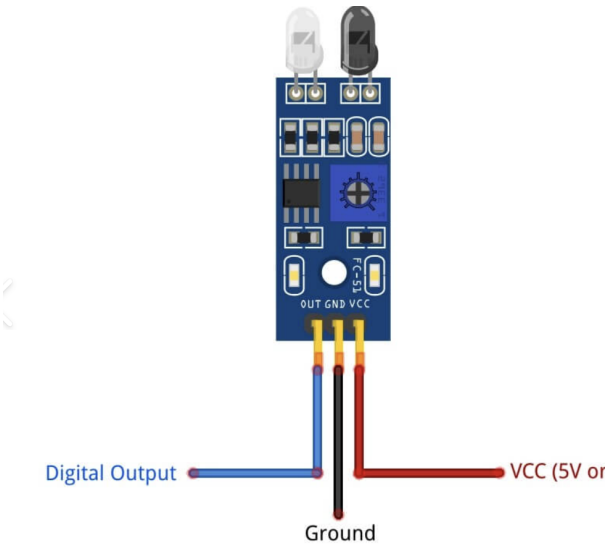

Figure 2: IR Reciever Pinout

The IR remote is used as the user input device and the IR receiver interprets the signals. When a button on the remote is pressed, an IR command is sent to the receiver. The receiver sends this information to the Arduino.

The IR receiver is connected to pin A0 on the microcontroller. Different buttons on the remote were mapped to have different operating modes. The slow, medium, and fast modes allow the user to wirelessly control the cat toy system.

2.1.1 Sensing Element Code

The code for the IR remote is shown in figure 3.

```
//creating function to check remote signal
void checkRemote() {
  //if a signal has been received by the remote
  if (IrReceiver.decode()) {
    //getting command value from the remote
    byte cmd = IrReceiver.decodedIRData.command;

    //checking which button was pressed and updaing mode accoringly
    if (cmd == CMD_OFF) {
      currentMode = MODE_OFF; //turn system off
    }

    else if (cmd == CMD_SLOW) {
      currentMode = MODE_SLOW;    //put system in slow mode
      moveDelay = 1800;           //larger delay between motions
    }
    else if (cmd == CMD_REGULAR) {
      currentMode = MODE_REGULAR; //system operating at medium speed
      moveDelay = 1000;           //slightly smaller delay
    }
    else if (cmd == CMD_FAST) {
      currentMode = MODE_FAST;    //sytem operating in fast mode
      moveDelay = 300;            //much shorter delay between movements
    }

    //get ready to receive the next signal
    IrReceiver.resume();
    delay(150);
  }
}
```

Figure 3: IR Remote Code

The checkRemote() function checked for an input signal from the IR remote, and then checked the signal with predefined values. If the predefined value matched the button press, the operating mode was chosen. Different buttons on the remote had different modes for the cat toy. If the button was pressed, a system would update and the delay would also be updated. Testing confirmed that the IR receiver could use the button presses to control the speed of the cat toy.

2.2 Movement Control

The movement control system for the laser cat toy system was created using two servo motors connected to the Arduino. The pinout for a servo motor is shown in figure 4.

Figure 4: Servo Motor Pinout

One servo motor is connected to pin 3 on the Arduino and the other is connected to pin 6. The servo motor connected to pin 6 controls the horizontal movement of the laser toy system, and the servo motor connected to pin 6 controls the vertical movement. Both servo motors together allow the laser to move in two directions.

The system includes three movement modes that are slow, medium, and fast. Each mode changes the timing of the laser movements. Slow mode uses longer delays and smaller movements, while fast mode has shorter delays and larger movements. The Arduino updates the servo position based on the millis() function. This lets the system respond while the laser is moving. Overall, the movement system allows the laser pointer to create randomized patterns to resemble prey like motion.

Testing confirmed that the servos were able to generate and move to a randomized position at the correct time intervals. Each of the operating modes were functional, with the slow mode allowing smaller motions over a larger interval of time and the fast mode allowing larger motions over a much shorter period of time.

2.2.1 Motion Control Code

The randomized motion of the servo motors was achieved first by generating a random seed in the setup section of the code. This was done using the signal from an unused analog pin. Randomizing the seed ensured that the motion of the servos were not repeated over several uses, keeping the motion pattern unique. This line of code is shown in Figure 5.

```
//random generator based on an unused pin
randomSeed(analogRead(A1));
```

Figure 5: Random Seed Generator Code

The motion of the servos was largely controlled with the moveLaser() function. The millis() function was used to keep track of the current time, and the servos would only move when the correct amount of time had passed, which was determined by the selected mode. Minimum motion values were determined for each of the modes of operation. This section of the code is shown in Figure 6.

```
//creating function to control the motion of the laser
void moveLaser() {
  //track the current time
  unsigned long currentTime = millis();

  //only move when the correct amount of time has passed
  if (currentTime - lastMoveTime >= (unsigned long)moveDelay) {

    //declare variables for new vertical and horizontal positions
    int newPan;
    int newTilt;

    //declare variable for minimum motion
    int minMove;

    //defining different movement sizes for each mode
    if (currentMode == MODE_FAST) {
      minMove = 45; //larger motion
    }
    else if (currentMode == MODE_SLOW) {
      minMove = 20; //smaller movements allowed
    }
    else {
      minMove = 30;
    }
```

Figure 6: Servo Control Function

Random positions were then generated for the servos using the random() function. The random value was found between the minimum and maximum motion values. This was done for both the left/right servo and the up/down servo. This new position values were then assigned to the servos using the servo.write() command. This section of the code is shown in Figure 7.

```
    //generate a new left/right position thats greater than the minimum movement
    do {
      newPan = random(panMin, panMax + 1);
    }
    while (abs(newPan - panPos) < minMove);
```
```
    //generate a new up/down position thats greater than the minimum movement
    do {
      newTilt = random(tiltMin, tiltMax + 1);
    }
    while (abs(newTilt - tiltPos) < minMove);

    //update positions
    panPos = newPan;
    tiltPos = newTilt;

    //move servo motors to new positions
    servo1.write(panPos);
    servo2.write(tiltPos);

    //update last motion time
    lastMoveTime = currentTime;
  }
}
```

Figure 7: Servo Control Function Continued

2.3 Putting the Systems Together

Once each subsystem was tested, they were combined into one complete system, figure 8 shows the entire system put together and figure 9 shows the circuit set up. The code continuously checks for remote input while controlling the laser movement. The system starts by setting the servos to a centered position. It then runs in a loop where it listens

for remote commands and updates the operating mode. Each mode controls how quickly the laser moves. With both servo motors stacked on top the system is capable of moving in a 2D plane. One servo control horizontal motion (x-direction, left to right), while the other controls vertical motion (y-direction, up and down).

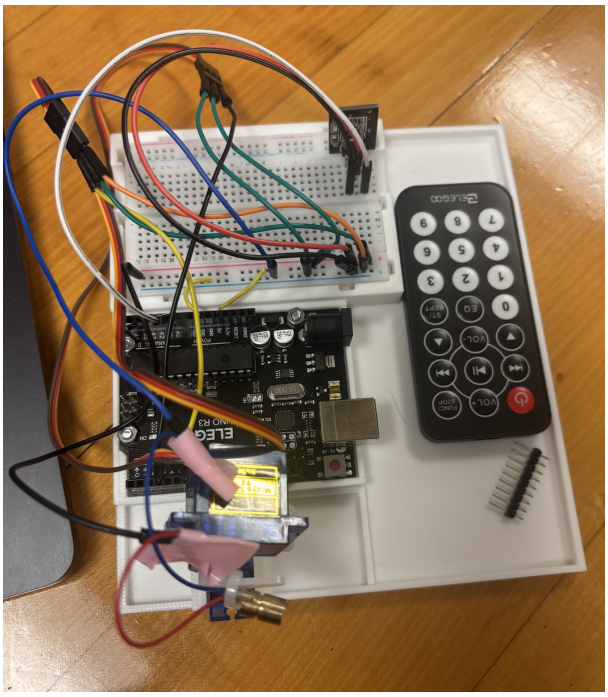

Figure 8: Full Circuit Setup

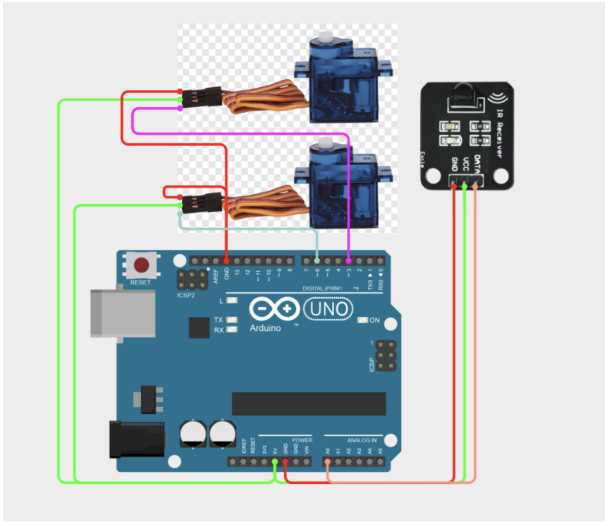

Figure 9: Full Circuit Schematic

New x and y positions are randomly generated within limits to keep motion within range, and reachable for the cat playing with the laser toy. A minimum movement distance is enforced so the laser does not make very small or jittery movements. Faster modes result in larger jumps and quicker movement. Slower modes create smaller, smoother motion. Both servos update at the same time, allowing the laser to move in any direction across the plane, including diagonally. This creates a more dynamic and less predictable

movement pattern. With all subsystems put together, the user can control the speed and behavior of the laser and have loads of fun with their cat!

3 Code

The code for the system was written in the Arduino IDE. The code started by calling in the libraries for the IR remote as well as the servo motors. The IR receiver pin as then defined as A0. The two servo motors were then attached to the correct digital pins of the Arduino. One of the servos controller the up and down motion of the system while the other controlled the left to right motion. Then the specific hexadecimal values corresponding to the buttons on the IR remote were defined for later use. These values allowed the program to identify which buttons had been pressed on the IR remote. This section of the code is shown in Figure 10.

```
//calling in libraries for IR remote and servo motors
#include <IRremote.h>
#include <Servo.h>

//defining IR receiver pin
const int RECV_PIN = A0;

//defining two servo motors
Servo servo1;   //controlls left to right
Servo servo2;   //controlls up dowm motion

//remote command values found from remote
const byte CMD_OFF     = 0x45;   //power button
const byte CMD_SLOW    = 0x0C;   //button 1
const byte CMD_REGULAR = 0x18;   //button 2
const byte CMD_FAST    = 0x5E;   //button 3
```

Figure 10: Start of System Code

Each of the operating states of the system were then defined using an enumeration. The four modes of the system were off, slow, medium, and fast. The variable "currentMode" was then defined to store the current operating mode of the system. The initial operating mode of the system was set to "off" so that the system would start in an inactive state. The variable "lastModeTime" was also created to track the time between movements of the servo motors and the "moveDelay" variable was created to control the time between movements. Several limits were then defined for the motions of the servo motors. Maximum and minimum values were created for both the horizontal and vertical servo motors. The starting positions of both motors were set to 90 degrees. This section of the code is shown in Figure 11.

```
//defining different opperating modes
//goal was three modes; slow, medium, and fast
enum Mode {
  MODE_OFF,
  MODE_REGULAR,
  MODE_FAST,
  MODE_SLOW
};

//creating variable to store current mode
//starting with system off
Mode currentMode = MODE_OFF;

//creating a variable to track the last time the laser moved
unsigned long lastMoveTime = 0;

//delay betweem movements
int moveDelay = 1000;

//setting limits to the range of motion of the motors
int panMin = 20;
int panMax = 160;
int tiltMin = 45;
int tiltMax = 135;

//defining starting position of the motors
int panPos = 90;
int tiltPos = 90;
```

Figure 11: System's Code Continued

A function was then created to check for a remote signal. This function checked for input from the IR remote, and compared the decoded signal to the predefined command values. If a match was found, the system's operating mode would be updated according to which button was pushed and the delay between servo motions would also be updated accordingly. For example, the slow mode corresponded to an increased delay of 1800 ms between motions while the fast mode corresponded to a much shorter delay of 300 ms. This section of the code is shown in Figure 12.

```
//creating function to check remote signal
void checkRemote() {
  //if a signal has been received by the remote
  if (IrReceiver.decode()) {
    //getting command value from the remote
    byte cmd = IrReceiver.decodedIRData.command;

    //checking which button was pressed and updaing mode accoringly
    if (cmd == CMD_OFF) {
      currentMode = MODE_OFF; //turn system off
    }

    else if (cmd == CMD_SLOW) {
      currentMode = MODE_SLOW;    //put system in slow mode
      moveDelay = 1800;           //larger delay between motions
    }
    else if (cmd == CMD_REGULAR) {
      currentMode = MODE_REGULAR; //system operating at medium speed
      moveDelay = 1000;           //slightly smaller delay
    }
    else if (cmd == CMD_FAST) {
      currentMode = MODE_FAST;    //sytem operating in fast mode
      moveDelay = 300;            //much shorter delay between movements
    }

    //get ready to receive the next signal
    IrReceiver.resume();
    delay(150);
  }
}
```

Figure 12: Remote Handling Function

Another function was defined to control the motion of the laser. This function first tracked the current time using millis() and checked to see if enough time had passed

between movements based on the selected motion delay time. If this condition was met, new positions for the horizontal and vertical servos would be generated. Additionally, a variable "minMove" was created to store the minimum required change in the positions of the servos. The value of this variable would change depending on the mode of the system, with smaller movements being allowed in the slow mode and larger movements being required in the fast mode. This allowed for the motion of the system to be more dynamic. Then, random positions were generated for each servo within the predefined limits. Once a valid position was generated, the positions of the servo's were update using the servo.write() function, and the "lastMoveTime" variable was updated to track the time of the most recent movement. This section of the code is shown in Figures 13 and 14.

```
//creating function to control the motion of the laser
void moveLaser() {
  //track the current time
  unsigned long currentTime = millis();

  //only move when the correct amount of time has passed
  if (currentTime - lastMoveTime >= (unsigned long)moveDelay) {

    //declare variables for new vertical and horizontal positions
    int newPan;
    int newTilt;

    //declare variable for minimum motion
    int minMove;

    //defining different movement sizes for each mode
    if (currentMode == MODE_FAST) {
      minMove = 45; //larger motion
    }
    else if (currentMode == MODE_SLOW) {
      minMove = 20; //smaller movements allowed
    }
    else {
      minMove = 30;
    }
```

Figure 13: Laser Control Function

```
    //generate a new left/right position thats greater than the minimum movement
    do {
      newPan = random(panMin, panMax + 1);
    }
    while (abs(newPan - panPos) < minMove);
```
```
    //generate a new up/down position thats greater than the minimum movement
    do {
      newTilt = random(tiltMin, tiltMax + 1);
    }
    while (abs(newTilt - tiltPos) < minMove);

    //update positions
    panPos = newPan;
    tiltPos = newTilt;

    //move servo motors to new positions
    servo1.write(panPos);
    servo2.write(tiltPos);

    //update last motion time
    lastMoveTime = currentTime;
  }
}
```

Figure 14: Laser Control Function Continued

To start the setup section of the code, serial communication was initialized. The servo motors were then assigned to their corresponding pins (3 and 6), and each servo was commanded to move into its initial position. The IR receiver was then initialized and a random seed was generated using an unused analog pin, allowing for the motion of the servos to be randomized. This section of the code is shown in Figure 15.

```
void setup() {
  //starting serial communication
  Serial.begin(9600);

  //assigning servos to the correct pins
  servo1.attach(3);
  servo2.attach(6);

  //moving the motors into their initial positions
  servo1.write(panPos);
  servo2.write(tiltPos);

  //initialize the IR receiver
  IrReceiver.begin(RECV_PIN, ENABLE_LED_FEEDBACK);

  //random generator based on an unused pin
  randomSeed(analogRead(A1));
}
```

Figure 15: Setup Section for System Code

In the main loop of the code, the program continually checked for input from the IR remote using the "checkRemote()" function. Then, if the system was not in "off mode" the "moveLaser()" function was called to change the function of the servo motors. This section of the code is shown in Figure 16.

```
void loop() {
  //check for remote input
  checkRemote();

  //only move the laser if the system is not in off mode
  if (currentMode != MODE_OFF) {
    moveLaser();
  }
}
```

Figure 16: Main Loop of System Code

4 Performance Overview

The laser cat toy successfully demonstrated the ability to control a laser pointer using two servo motors and an Arduino Uno. The IR remote was able to switch the system between slow, medium, and fast moves. Each mode had different movement speeds and all had randomized patterns.

During testing, the servo motors responded to the position commands. The use of the timing function allowed the system to work without long delays. The fast mode had quicker larger movements, while the slow mode had slower smaller movements. The medium mode was in between these two.

Overall, the system met the general project requirements by integrating IR communication, servo motor control, random motion patterns, and different control modes.

5 GitHub

A Git Hub link was created for this project. The repository contains all of the code, images, and documentation required to understand this project. The link to the Git Hub

repository is found below. https://github.com/shealynkeelen/Cat$_L aser$

6 Conclusion/Reflection

Overall, the laser cat toy project showed the integration of different components into one system. The project combined IR remote control, servo motor control, and randomized laser movement. The system could operate in different modes and respond to user input. Testing showed that the system functioned as intended, with each movement creating different patterns. The use of randomized movement made the toy more interactive for a cat.

If more time were available, one improvement that could be made would be improving the overall appearance of the toy. While the system functioned as intended, creating a better mounting system could make the project look more visually appealing. Overall, this project successfully combined movement control, IR communication, and Arduino programming to create an interactive and functional laser cat toy system.

7 Full Written Code

```
//calling in libraries for IR remote and servo motors
#include <IRremote.h>
#include <Servo.h>

//defining IR receiver pin
const int RECV_PIN = A0;

//defining two servo motors
Servo servo1;   //controlls left to right
Servo servo2;   //controlls up dowm motion

//remote command values found from remote
const byte CMD_OFF     = 0x45;   //power button
const byte CMD_SLOW    = 0x0C;   //button 1
const byte CMD_REGULAR = 0x18;   //button 2
const byte CMD_FAST    = 0x5E;   //button 3

//defining different opperating modes
//goal was three modes; slow, medium, and fast
enum Mode {
  MODE_OFF,
  MODE_REGULAR,
  MODE_FAST,
  MODE_SLOW
};

//creating variable to store current mode
//starting with system off
Mode currentMode = MODE_OFF;
```

```
//creating a variable to track the last time the laser moved
unsigned long lastMoveTime = 0;

//delay betweem movements
int moveDelay = 1000;

//setting limits to the range of motion of the motors
int panMin = 20;
int panMax = 160;
int tiltMin = 45;
int tiltMax = 135;

//defining starting position of the motors
int panPos = 90;
int tiltPos = 90;

//creating function to check remote signal
void checkRemote() {
  //if a signal has been received by the remote
  if (IrReceiver.decode()) {
    //getting command value from the remote
    byte cmd = IrReceiver.decodedIRData.command;

    //checking which button was pressed and updaing mode accoringly
    if (cmd == CMD_OFF) {
      currentMode = MODE_OFF; //turn system off
    }

    else if (cmd == CMD_SLOW) {
      currentMode = MODE_SLOW;    //put system in slow mode
      moveDelay = 1800;           //larger delay between motions
    }
    else if (cmd == CMD_REGULAR) {
      currentMode = MODE_REGULAR; //system operating at medium speed
      moveDelay = 1000;           //slightly smaller delay
    }
    else if (cmd == CMD_FAST) {
      currentMode = MODE_FAST;    //sytem operating in fast mode
      moveDelay = 300;            //much shorter delay between movements
    }

    //get ready to receive the next signal
    IrReceiver.resume();
    delay(150);
  }
}
```

```
//creating function to control the motion of the laser
void moveLaser() {
  //track the current time
  unsigned long currentTime = millis();

  //only move when the correct amount of time has passed
  if (currentTime - lastMoveTime >= (unsigned long)moveDelay) {

    //declare variables for new vertical and horizontal positions
    int newPan;
    int newTilt;

    //declare variable for minimum motion
    int minMove;

    //defining different movement sizes for each mode
    if (currentMode == MODE_FAST) {
      minMove = 45; //larger motion
    }
    else if (currentMode == MODE_SLOW) {
      minMove = 20; //smaller movements allowed
    }
    else {
      minMove = 30;
    }

    //generate a new left/right position thats greater than the minimum movement
    do {
      newPan = random(panMin, panMax + 1);
    }
    while (abs(newPan - panPos) < minMove);
```
```
    //generate a new up/down position thats greater than the minimum movement
    do {
      newTilt = random(tiltMin, tiltMax + 1);
    }
    while (abs(newTilt - tiltPos) < minMove);

    //update positions
    panPos = newPan;
    tiltPos = newTilt;

    //move servo motors to new positions
    servo1.write(panPos);
    servo2.write(tiltPos);

    //update last motion time
    lastMoveTime = currentTime;
```

```
  }
}

void setup() {
  //starting serial communication
  Serial.begin(9600);

  //assigning servos to the correct pins
  servo1.attach(3);
  servo2.attach(6);

  //moving the motors into their initial positions
  servo1.write(panPos);
  servo2.write(tiltPos);

  //initialize the IR receiver
  IrReceiver.begin(RECV_PIN, ENABLE_LED_FEEDBACK);

  //random generator based on an unused pin
  randomSeed(analogRead(A1));
}

void loop() {
  //check for remote input
  checkRemote();

  //only move the laser if the system is not in off mode
  if (currentMode != MODE_OFF) {
    moveLaser();
  }
}
```

8 References

https://arduinoyard.com/ir-sensor-with-esp32/ https://lastminuteengineers.com/servo-motor-arduino-tutorial/